

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1102

COURSE OUTCOME COVERED

CO5: Analyze object-oriented software using quality assurance and usability testing
Techniques (BL4, BL5)
Assessment Tool Weightage: 25%

QUESTIONS: 05

Total Marks:25

Annexure A

Error Analysis Task

Code of Conduct:

*I E.Ragul / 192571032 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

E. Ragul

Annexure A

Error Analysis Task

Q1. Scenario: Online Banking System

Modules:

User Login, Fund Transfer, Account Verification

Question:

Write pseudocode for a quality assurance testing process that validates secure user authentication and fund transfer operations. Include input validation, transaction verification, exception handling, and test result logging.

Q2. Scenario: E-Commerce Shopping Platform

Features:

Product Search, Shopping Cart, Payment Gateway

Question:

Design pseudocode for a usability testing evaluation that measures customer interaction efficiency, including product search time, checkout completion time, and navigation complexity. Explain how the algorithm collects and analyzes user behavior data.

Q3. Scenario: Hospital Management System

Modules:

Patient Registration, Appointment Scheduling, Medical Records

Question:

Write pseudocode to generate and execute test cases for appointment scheduling functionality. Include validation of doctor availability, appointment conflicts, patient information accuracy, and system responses to invalid requests.

Q4. Scenario: Airline Reservation System

Modules:

Passenger Registration, Flight Booking, Seat Allocation

Question:

Develop pseudocode for a constraint-based testing approach where booking requests must satisfy conditions such as valid passenger information, seat availability, and payment confirmation. Show how invalid booking scenarios are detected and handled.

Q5. Scenario: Smart Library Management System

Features:

Book Search, Book Reservation, Borrow/Return Services

Question:

Write pseudocode for integrating quality assurance and usability testing where the system evaluates both functional correctness (reservation and borrowing operations) and user experience factors such as response time, ease of navigation, and accessibility.

Annexure A — Error Analysis Task

Q1. Online Banking System — Secure Authentication & Fund Transfer QA Testing

Scenario: The pseudocode below designs a QA testing process covering the User Login, Fund Transfer, and Account Verification modules. It validates inputs, verifies transactions, handles exceptions, and logs every test result for audit and defect-tracking purposes.

```
BEGIN QA_TestBankingSystem

INITIALIZE testLog AS empty list
INITIALIZE testCaseID = 1000

// ----- MODULE 1: Input Validation -----
FUNCTION ValidateLoginInput(username, password)
  IF username IS EMPTY OR password IS EMPTY THEN
    RETURN FAIL, "Empty credentials"
  IF LENGTH(password) < 8 THEN
    RETURN FAIL, "Password below minimum length"
  IF NOT MATCHES_REGEX(username, VALID_USERNAME_PATTERN) THEN
    RETURN FAIL, "Invalid username format"
  RETURN PASS, "Input format valid"
END FUNCTION

// ----- MODULE 2: Authentication Test -----
FUNCTION TestUserLogin(username, password, expectedResult)
  TRY
    (status, msg) = ValidateLoginInput(username, password)
    IF status == FAIL THEN
      LogResult(testCaseID, "Login-InputValidation", FAIL, msg)
      RETURN
    END IF

    response = AuthenticationService.Login(username, password)

    IF response.status == expectedResult THEN
      LogResult(testCaseID, "Login-Auth", PASS,
        "Expected " + expectedResult + " achieved")
    ELSE
      LogResult(testCaseID, "Login-Auth", FAIL,
        "Expected " + expectedResult + " got " + response.status)
    END IF

    // Lock account after repeated failures (security test)
    IF response.status == "INVALID_CREDENTIALS" THEN
      attemptCount = AuthenticationService.GetFailedAttempts(username)
      IF attemptCount >= MAX_ATTEMPTS THEN
        ASSERT AuthenticationService.IsAccountLocked(username) == TRUE
        LogResult(testCaseID, "Login-LockoutPolicy", PASS, "Account locked as expected")
      END IF
    END IF
  END IF
```

```

CATCH ConnectionException AS e
    LogResult(testCaseID, "Login-Auth", ERROR, "Service unreachable: " + e.message)
CATCH TimeoutException AS e
    LogResult(testCaseID, "Login-Auth", ERROR, "Request timed out: " + e.message)
FINALLY
    testCaseID = testCaseID + 1
END TRY
END FUNCTION

// ----- MODULE 3: Fund Transfer & Account Verification -----
FUNCTION TestFundTransfer(session, sourceAcct, destAcct, amount)
    TRY
        // Input validation
        IF amount <= 0 THEN
            LogResult(testCaseID, "Transfer-InputValidation", FAIL, "Non-positive amount rejected as expected")
            RETURN
        END IF
        IF sourceAcct = destAcct THEN
            LogResult(testCaseID, "Transfer-InputValidation", FAIL, "Self-transfer flagged as expected")
            RETURN
        END IF

        // Account verification
        IF NOT AccountService.Exists(sourceAcct) OR NOT AccountService.Exists(destAcct) THEN
            LogResult(testCaseID, "Transfer-AccountVerification", FAIL, "Unknown account rejected as expected")
            RETURN
        END IF

        balanceBefore = AccountService.GetBalance(sourceAcct)
        IF balanceBefore < amount THEN
            LogResult(testCaseID, "Transfer-BalanceCheck", PASS, "Insufficient funds correctly blocked")
            RETURN
        END IF

        // Execute transaction
        transactionID = TransferService.Execute(session, sourceAcct, destAcct, amount)
        balanceAfter = AccountService.GetBalance(sourceAcct)

        // Transaction verification (state + ledger consistency)
        expectedBalance = balanceBefore - amount
        IF balanceAfter == expectedBalance
            AND LedgerService.HasEntry(transactionID)
            AND AccountService.GetBalance(destAcct) increased by amount THEN
                LogResult(testCaseID, "Transfer-Execution", PASS, "Transaction " + transactionID + " verified")
            ELSE
                LogResult(testCaseID, "Transfer-Execution", FAIL, "Balance/ledger mismatch after transfer")
            END IF
        ELSE
            LogResult(testCaseID, "Transfer-Execution", FAIL, "Balance/ledger mismatch after transfer")
        END IF

        CATCH InsufficientFundsException AS e
            LogResult(testCaseID, "Transfer-Execution", ERROR, e.message)
        CATCH TransactionRollbackException AS e
            LogResult(testCaseID, "Transfer-Execution", ERROR, "Rollback triggered: " + e.message)
        CATCH GenericException AS e
            LogResult(testCaseID, "Transfer-Execution", ERROR, "Unhandled error: " + e.message)
        FINALLY
    END TRY
END FUNCTION

```

```

        testCaseID = testCaseID + 1
    END TRY
END FUNCTION

// ----- MODULE 4: Result Logging -----
FUNCTION LogResult(id, testName, verdict, details)
    entry = { ID: id, Name: testName, Result: verdict,
              Details: details, Timestamp: CURRENT_TIMESTAMP() }
    APPEND entry TO testLog
    WRITE entry TO TestReport.csv
    IF verdict == ERROR THEN ALERT QA_Team(entry) // escalate system errors
END FUNCTION

// ----- Driver -----
TestUserLogin("validUser1", "Str0ngP@ss!", "SUCCESS")
TestUserLogin("validUser1", "wrongPass", "INVALID_CREDENTIALS")
session = AuthenticationService.Login("validUser1", "Str0ngP@ss!")
TestFundTransfer(session, "ACC1001", "ACC2002", 500.00)
TestFundTransfer(session, "ACC1001", "ACC2002", -50.00) // negative-amount edge case
PRINT "Total tests executed: " + LENGTH(testLog)

END QA_TestBankingSystem

```

Q2. E-Commerce Shopping Platform — Usability Testing

Scenario: The pseudocode below designs a usability evaluation for the Product Search, Shopping Cart, and Payment Gateway features. It times key user tasks, records interaction events, and computes navigation-complexity and efficiency metrics from the collected behaviour data.

```

BEGIN UsabilityTest_ECommerce

    INITIALIZE sessionLog AS empty list
    INITIALIZE participants = LOAD_TEST_PARTICIPANTS()

    FOR EACH participant IN participants DO
        session = StartUserSession(participant)
        metrics = { searchTime: NULL, checkoutTime: NULL,
                    clickCount: 0, backNavigations: 0, errorsEncountered: 0 }

        TRY
            // ---- Task 1: Product Search ----
            searchStart = CURRENT_TIME()
            RecordEvent(session, "SEARCH_INITIATED")
            results = ProductSearch.Query(participant.searchTerm)
            metrics.clickCount += CountClicksUntil(results)

            IF results IS EMPTY THEN
                metrics.errorsEncountered += 1
                RecordEvent(session, "SEARCH_ZERO_RESULTS")
            END IF
            searchEnd = CURRENT_TIME()
        CATCH
            RecordEvent(session, "SEARCH_ERROR")
        END TRY
    END FOR
END

```

```

metrics.searchTime = searchEnd - searchStart

// ---- Task 2: Shopping Cart ----
selectedItem = participant.SelectFromResults(results)
Cart.AddItem(session, selectedItem)
IF Cart.Contains(session, selectedItem) != TRUE THEN
    metrics.errorsEncountered += 1
    RecordEvent(session, "CART_ADD_FAILED")
END IF

// ---- Task 3: Checkout / Payment Gateway ----
checkoutStart = CURRENT_TIME()
navPathLength = 0
WHILE session.currentPage != "ORDER_CONFIRMATION" DO
    action = participant.NextAction(session)
    IF action == "BACK" THEN
        metrics.backNavigations += 1
    END IF
    session = NavigateTo(session, action)
    navPathLength += 1
    IF navPathLength > MAX_EXPECTED_STEPS THEN
        metrics.errorsEncountered += 1
        RecordEvent(session, "EXCESSIVE_NAVIGATION")
        BREAK
    END IF
END WHILE

paymentResult = PaymentGateway.Process(session, participant.paymentInfo)
checkoutEnd = CURRENT_TIME()
metrics.checkoutTime = checkoutEnd - checkoutStart

IF paymentResult.status != "SUCCESS" THEN
    metrics.errorsEncountered += 1
    RecordEvent(session, "PAYMENT_FAILED: " + paymentResult.reason)
END IF

CATCH SessionTimeoutException AS e
    metrics.errorsEncountered += 1
    RecordEvent(session, "SESSION_TIMEOUT: " + e.message)
CATCH UIElementNotFoundException AS e
    metrics.errorsEncountered += 1
    RecordEvent(session, "UI_ERROR: " + e.message)
FINALLY
    navigationComplexity = metrics.clickCount + (2 * metrics.backNavigations)
    APPEND {participant: participant.id, metrics: metrics,
           navigationComplexity: navigationComplexity} TO sessionLog
    EndUserSession(session)
END TRY
END FOR

// ---- Aggregate Analysis of Collected Behaviour Data ----
avgSearchTime = AVERAGE(sessionLog[*].metrics.searchTime)
avgCheckoutTime = AVERAGE(sessionLog[*].metrics.checkoutTime)
avgNavComplexity = AVERAGE(sessionLog[*].navigationComplexity)
taskSuccessRate = COUNT(sessionLog WHERE metrics.errorsEncountered == 0) / LENGTH(sessionLog)

```

```

PRINT "Avg Search Time: " + avgSearchTime + "s"
PRINT "Avg Checkout Time: " + avgCheckoutTime + "s"
PRINT "Avg Navigation Complexity Score: " + avgNavComplexity
PRINT "Task Success Rate: " + (taskSuccessRate * 100) + "%"

IF avgSearchTime > USABILITY_THRESHOLD_SEARCH
  OR avgCheckoutTime > USABILITY_THRESHOLD_CHECKOUT THEN
  FLAG "Usability issue: task times exceed acceptable thresholds"
END IF

WRITE sessionLog TO UsabilityReport.csv

END UsabilityTest_ECommerce

```

How the algorithm collects and analyses behaviour data: each participant's session is instrumented with timestamped event logging (RecordEvent) at every meaningful interaction — search submission, cart update, page navigation, and payment attempt. Task time is measured as the delta between the start and end timestamps of each task, click count and back-navigations are tallied per session to derive a navigation-complexity score, and every anomaly (zero search results, failed cart add, payment failure, excessive navigation) increments an error counter. After all sessions complete, the algorithm aggregates these per-session metrics into averages and a task-success rate, then flags the interface if either task time exceeds a predefined usability threshold — turning raw interaction logs into actionable usability findings.

Q3. Hospital Management System — Appointment Scheduling Test Case Generation

Scenario: The pseudocode below generates and executes test cases for the Appointment Scheduling module, validating doctor availability, appointment conflicts, patient information accuracy, and the system's handling of invalid requests.

```

BEGIN TestAppointmentScheduling

  INITIALIZE testCases AS empty list
  INITIALIZE testLog AS empty list

  // ----- Test Case Generation -----
  FUNCTION GenerateTestCases()
    ADD {id:"TC01", type:"VALID", patient:VALID_PATIENT, doctor:AVAILABLE_DOCTOR,
      slot:FREE_SLOT} TO testCases
    ADD {id:"TC02", type:"DOCTOR_UNAVAILABLE", patient:VALID_PATIENT,
      doctor:ON_LEAVE_DOCTOR, slot:ANY_SLOT} TO testCases
    ADD {id:"TC03", type:"SLOT_CONFLICT", patient:VALID_PATIENT, doctor:AVAILABLE_DOCTOR,
      slot:ALREADY_BOOKED_SLOT} TO testCases
    ADD {id:"TC04", type:"INVALID_PATIENT_INFO", patient:PATIENT_MISSING_ID,
      doctor:AVAILABLE_DOCTOR, slot:FREE_SLOT} TO testCases
    ADD {id:"TC05", type:"PAST_DATE_REQUEST", patient:VALID_PATIENT,
      doctor:AVAILABLE_DOCTOR, slot:PAST_DATE_SLOT} TO testCases
    ADD {id:"TC06", type:"DUPLICATE_BOOKING", patient:PATIENT_WITH_EXISTING_APPT,
      doctor:AVAILABLE_DOCTOR, slot:FREE_SLOT} TO testCases
  RETURN testCases

```

```

END FUNCTION

// ----- Validation Helpers -----
FUNCTION ValidatePatientInfo(patient)
  IF patient.id IS NULL OR patient.name IS EMPTY THEN
    RETURN FALSE, "Missing mandatory patient details"
  IF NOT MATCHES_FORMAT(patient.contactNumber, PHONE_PATTERN) THEN
    RETURN FALSE, "Invalid contact number format"
  RETURN TRUE, "Patient information valid"
END FUNCTION

FUNCTION CheckDoctorAvailability(doctor, slot)
  RETURN DoctorSchedule.IsFree(doctor, slot) AND NOT DoctorSchedule.IsOnLeave(doctor, slot.date)
END FUNCTION

FUNCTION CheckSlotConflict(doctor, slot)
  RETURN AppointmentDB.Exists(doctor, slot) // TRUE = conflict
END FUNCTION

// ----- Test Execution Engine -----
FUNCTION ExecuteTestCase(tc)
  TRY
    (isValidPatient, patientMsg) = ValidatePatientInfo(tc.patient)
    IF NOT isValidPatient THEN
      RESULT = (tc.type == "INVALID_PATIENT_INFO") ? PASS : FAIL
      LogResult(tc.id, RESULT, patientMsg)
      RETURN
    END IF

    IF tc.slot.date < CURRENT_DATE() THEN
      RESULT = (tc.type == "PAST_DATE_REQUEST") ? PASS : FAIL
      LogResult(tc.id, RESULT, "Past-date slot correctly rejected")
      RETURN
    END IF

    IF NOT CheckDoctorAvailability(tc.doctor, tc.slot) THEN
      RESULT = (tc.type == "DOCTOR_UNAVAILABLE") ? PASS : FAIL
      LogResult(tc.id, RESULT, "Doctor unavailable — booking correctly blocked")
      RETURN
    END IF

    IF CheckSlotConflict(tc.doctor, tc.slot) THEN
      RESULT = (tc.type == "SLOT_CONFLICT") ? PASS : FAIL
      LogResult(tc.id, RESULT, "Double-booking correctly prevented")
      RETURN
    END IF

    IF PatientHasAppointment(tc.patient, tc.slot.date) AND tc.type == "DUPLICATE_BOOKING" THEN
      LogResult(tc.id, PASS, "Duplicate same-day booking correctly flagged")
      RETURN
    END IF

    // All checks passed — attempt actual booking
    appointmentID = AppointmentService.Book(tc.patient, tc.doctor, tc.slot)
    IF appointmentID IS NOT NULL AND tc.type == "VALID" THEN

```

```

        LogResult(tc.id, PASS, "Appointment " + appointmentID + " booked successfully")
    ELSE
        LogResult(tc.id, FAIL, "Unexpected booking outcome for case type " + tc.type)
    END IF

    CATCH SchedulingConflictException AS e
        LogResult(tc.id, (tc.type != "VALID") ? PASS : ERROR, e.message)
    CATCH DatabaseException AS e
        LogResult(tc.id, ERROR, "DB failure: " + e.message)
    END TRY
END FUNCTION

FUNCTION LogResult(id, verdict, details)
    APPEND {TestCase: id, Result: verdict, Details: details,
        Time: CURRENT_TIMESTAMP()} TO testLog
END FUNCTION

// ----- Driver -----
cases = GenerateTestCases()
FOR EACH tc IN cases DO
    ExecuteTestCase(tc)
END FOR

PRINT "Pass: " + COUNT(testLog WHERE Result == PASS)
PRINT "Fail: " + COUNT(testLog WHERE Result == FAIL)
PRINT "Error: " + COUNT(testLog WHERE Result == ERROR)
WRITE testLog TO SchedulingTestReport.csv

END TestAppointmentScheduling

```

Q4. Airline Reservation System — Constraint-Based Booking Testing

Scenario: The pseudocode below implements constraint-based testing for the Passenger Registration, Flight Booking, and Seat Allocation modules. A booking request must simultaneously satisfy passenger-information validity, seat availability, and payment confirmation constraints; the algorithm evaluates each constraint independently and detects the specific point of failure for invalid booking scenarios.

```

BEGIN ConstraintBasedTest_AirlineBooking

    DEFINE constraints = [
        "C1_PASSENGER_INFO_VALID",
        "C2_FLIGHT_EXISTS",
        "C3_SEAT_AVAILABLE",
        "C4_NO_DUPLICATE_SEAT_ASSIGNMENT",
        "C5_PAYMENT_CONFIRMED"
    ]

    INITIALIZE testLog AS empty list

    // ----- Individual Constraint Checks -----

```

```

FUNCTION CheckPassengerInfo(passenger)
  IF passenger.name IS EMPTY OR passenger.passportNumber IS EMPTY THEN
    RETURN FALSE, "C1_PASSENGER_INFO_VALID"
  IF passenger.age < 0 OR passenger.age > 120 THEN
    RETURN FALSE, "C1_PASSENGER_INFO_VALID"
  RETURN TRUE, NULL
END FUNCTION

FUNCTION CheckFlightExists(flightID)
  IF NOT FlightDB.Exists(flightID) THEN
    RETURN FALSE, "C2_FLIGHT_EXISTS"
  RETURN TRUE, NULL
END FUNCTION

FUNCTION CheckSeatAvailability(flightID, seatClass)
  availableSeats = FlightDB.GetAvailableSeats(flightID, seatClass)
  IF availableSeats <= 0 THEN
    RETURN FALSE, "C3_SEAT_AVAILABLE"
  RETURN TRUE, NULL
END FUNCTION

FUNCTION CheckSeatNotAlreadyAssigned(flightID, seatNumber)
  IF SeatMap.IsOccupied(flightID, seatNumber) THEN
    RETURN FALSE, "C4_NO_DUPLICATE_SEAT_ASSIGNMENT"
  RETURN TRUE, NULL
END FUNCTION

FUNCTION CheckPaymentConfirmed(paymentInfo, fareAmount)
  result = PaymentGateway.Authorize(paymentInfo, fareAmount)
  IF result.status != "CONFIRMED" THEN
    RETURN FALSE, "C5_PAYMENT_CONFIRMED"
  RETURN TRUE, NULL
END FUNCTION

// ----- Constraint Solver / Test Driver -----
FUNCTION ProcessBookingRequest(request)
  violatedConstraints = empty list

  TRY
    (ok, failedC) = CheckPassengerInfo(request.passenger)
    IF NOT ok THEN APPEND failedC TO violatedConstraints

    (ok, failedC) = CheckFlightExists(request.flightID)
    IF NOT ok THEN
      APPEND failedC TO violatedConstraints
    ELSE
      (ok, failedC) = CheckSeatAvailability(request.flightID, request.seatClass)
      IF NOT ok THEN APPEND failedC TO violatedConstraints

      (ok, failedC) = CheckSeatNotAlreadyAssigned(request.flightID, request.seatNumber)
      IF NOT ok THEN APPEND failedC TO violatedConstraints
    END IF

    // Payment is only attempted if all prior constraints hold —
    // avoids charging a passenger for an invalid/unavailable booking

```

```

IF LENGTH(violatedConstraints) == 0 THEN
    (ok, failedC) = CheckPaymentConfirmed(request.paymentInfo, request.fareAmount)
    IF NOT ok THEN APPEND failedC TO violatedConstraints
END IF

IF LENGTH(violatedConstraints) == 0 THEN
    bookingID = ReservationService.ConfirmBooking(request)
    SeatMap.Assign(request.flightID, request.seatNumber, request.passenger)
    LogResult(request.id, PASS, "All constraints satisfied — Booking " + bookingID + " confirmed")
ELSE
    expectedInvalid = (request.expectedOutcome == "INVALID")
    RESULT = expectedInvalid ? PASS : FAIL
    LogResult(request.id, RESULT,
        "Booking correctly rejected — violated: " + JOIN(violatedConstraints, ", "))
END IF

CATCH PaymentGatewayException AS e
    LogResult(request.id, ERROR, "Payment service error: " + e.message)
CATCH ConcurrencyException AS e
    // Two requests raced for the same seat
    LogResult(request.id, ERROR, "Seat allocation race condition: " + e.message)
CATCH GenericException AS e
    LogResult(request.id, ERROR, "Unhandled exception: " + e.message)
END TRY
END FUNCTION

FUNCTION LogResult(id, verdict, details)
    APPEND {RequestID: id, Result: verdict, Details: details,
        Time: CURRENT_TIMESTAMP()} TO testLog
END FUNCTION

// ----- Driver: sample booking requests -----
ProcessBookingRequest({id:"BK01", passenger:VALID_PASSENGER, flightID:"AI202",
    seatClass:"ECONOMY", seatNumber:"14C", paymentInfo:VALID_CARD,
    fareAmount:5400, expectedOutcome:"VALID"})

ProcessBookingRequest({id:"BK02", passenger:PASSENGER_NO_PASSPORT, flightID:"AI202",
    seatClass:"ECONOMY", seatNumber:"14D", paymentInfo:VALID_CARD,
    fareAmount:5400, expectedOutcome:"INVALID"})

ProcessBookingRequest({id:"BK03", passenger:VALID_PASSENGER, flightID:"AI202",
    seatClass:"ECONOMY", seatNumber:"14C", // already assigned in BK01
    paymentInfo:VALID_CARD, fareAmount:5400, expectedOutcome:"INVALID"})

PRINT "Pass: " + COUNT(testLog WHERE Result == PASS)
PRINT "Fail: " + COUNT(testLog WHERE Result == FAIL)
WRITE testLog TO BookingConstraintReport.csv

END ConstraintBasedTest_AirlineBooking

```

Q5. Smart Library Management System — Integrated QA + Usability Testing

Scenario: The pseudocode below combines functional QA testing (Book Search, Book Reservation, Borrow/Return operations) with usability evaluation (response time, navigation ease, accessibility) in a single integrated test pass, so that each test case yields both a correctness verdict and a user-experience score.

```
BEGIN IntegratedTest_SmartLibrary

    INITIALIZE testLog AS empty list
    INITIALIZE ACCEPTABLE_RESPONSE_TIME = 2.0 // seconds
    INITIALIZE MAX_ACCEPTABLE_STEPS = 4 // navigation-ease threshold

    // ----- Functional QA: Book Search -----
    FUNCTION TestBookSearch(query, expectedFound)
        TRY
            startTime = CURRENT_TIME()
            results = LibrarySystem.SearchBooks(query)
            responseTime = CURRENT_TIME() - startTime

            functionalPass = (LENGTH(results) > 0) == expectedFound
            usabilityPass = responseTime <= ACCEPTABLE_RESPONSE_TIME

            LogResult("Search:" + query, functionalPass, usabilityPass,
                "responseTime=" + responseTime + "s, resultsFound=" + LENGTH(results))
        CATCH SearchIndexException AS e
            LogResult("Search:" + query, ERROR, ERROR, e.message)
        END TRY
    END FUNCTION

    // ----- Functional QA: Book Reservation -----
    FUNCTION TestBookReservation(user, bookID)
        TRY
            startTime = CURRENT_TIME()
            navSteps = 0

            IF NOT LibrarySystem.BookExists(bookID) THEN
                LogResult("Reserve:" + bookID, TRUE, TRUE, "Non-existent book correctly rejected")
                RETURN
            END IF

            WHILE LibrarySystem.CurrentPage(user) != "RESERVATION_CONFIRMED" DO
                LibrarySystem.NavigateStep(user)
                navSteps += 1
                IF navSteps > MAX_ACCEPTABLE_STEPS THEN BREAK
            END WHILE
            responseTime = CURRENT_TIME() - startTime

            copiesAvailable = LibrarySystem.GetAvailableCopies(bookID)
            IF copiesAvailable > 0 THEN
                reservationID = LibrarySystem.Reserve(user, bookID)
```

```

        functionalPass = (reservationID IS NOT NULL)
    ELSE
        waitlistPos = LibrarySystem.AddToWaitlist(user, bookID)
        functionalPass = (waitlistPos IS NOT NULL)
    END IF

    usabilityPass = (navSteps <= MAX_ACCEPTABLE_STEPS) AND
        (responseTime <= ACCEPTABLE_RESPONSE_TIME)

    LogResult("Reserve:" + bookID, functionalPass, usabilityPass,
        "navSteps=" + navSteps + ", responseTime=" + responseTime + "s")

    CATCH ReservationConflictException AS e
        LogResult("Reserve:" + bookID, ERROR, ERROR, e.message)
    END TRY
END FUNCTION

// ----- Functional QA: Borrow / Return -----
FUNCTION TestBorrowReturn(user, bookID)
    TRY
        // Borrow
        borrowResult = LibrarySystem.BorrowBook(user, bookID)
        borrowOK = (borrowResult.status == "SUCCESS") OR
            (borrowResult.status == "LIMIT_EXCEEDED" AND user.activeLoans >= MAX_LOANS)

        // Return (skip if borrow legitimately failed due to limit)
        IF borrowResult.status == "SUCCESS" THEN
            returnResult = LibrarySystem.ReturnBook(user, bookID)
            returnOK = (returnResult.status == "SUCCESS") AND
                (LibrarySystem.GetAvailableCopies(bookID) increased by 1)
        ELSE
            returnOK = TRUE // not applicable
        END IF

        // Accessibility check — screen-reader labels & keyboard navigation present
        accessibilityPass = UIAccessibilityChecker.Validate("BorrowReturnScreen")

        LogResult("BorrowReturn:" + bookID, (borrowOK AND returnOK),
            accessibilityPass, "borrow=" + borrowResult.status)

        CATCH OverdueBlockException AS e
            LogResult("BorrowReturn:" + bookID, TRUE, TRUE, "Overdue fine block enforced: " + e.message)
        CATCH DatabaseException AS e
            LogResult("BorrowReturn:" + bookID, ERROR, ERROR, e.message)
        END TRY
    END FUNCTION

// ----- Combined Logging -----
FUNCTION LogResult(caseName, functionalVerdict, usabilityVerdict, details)
    APPEND {Case: caseName, Functional: functionalVerdict,
        Usability: usabilityVerdict, Details: details,
        Time: CURRENT_TIMESTAMP()} TO testLog
END FUNCTION

// ----- Driver -----

```

```

TestBookSearch("Data Structures", TRUE)
TestBookSearch("zzz_nonexistent_title", FALSE)
TestBookReservation(user1, "BK-1001")
TestBookReservation(user1, "BK-9999") // invalid book ID
TestBorrowReturn(user1, "BK-1001")

functionalPassRate = COUNT(testLog WHERE Functional == TRUE) / LENGTH(testLog)
usabilityPassRate = COUNT(testLog WHERE Usability == TRUE) / LENGTH(testLog)

PRINT "Functional Pass Rate: " + (functionalPassRate * 100) + "%"
PRINT "Usability Pass Rate: " + (usabilityPassRate * 100) + "%"
WRITE testLog TO LibraryIntegratedTestReport.csv

END IntegratedTest_SmartLibrary

```

RUBRICS FOR CALCULATION:

Error Analysis Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand